Guida Introduttiva al Debugger GDB (GNU Debugger)

1 Cos'è un Debugger?

Spesso, quando un programma C presenta un comportamento anomalo o va in crash (es. *Segmentation fault*), la prima reazione istintiva è riempire il codice di `printf` per capire dove si blocchi l'esecuzione o quali siano i valori delle variabili.

Sebbene le `printf` possano aiutare, esiste uno strumento professionale e molto più potente: il **Debugger**. Un debugger è un programma che ti permette di "entrare" dentro la pancia del tuo codice mentre è in esecuzione, consentendoti di:

- Fermare il programma esattamente sulla riga di codice desiderata (impostando dei *breakpoint*).
- Ispezionare il valore di tutte le variabili in tempo reale.
- Eseguire il codice riga per riga per osservare passo dopo passo come evolve lo stato della memoria.
- Capire l'esatta istruzione che ha causato un crash improvviso.

2 Cos'è GDB?

`gdb` (GNU Debugger) è il debugger standard per i sistemi operativi Unix/Linux e C/C++. È uno strumento a riga di comando integrato con il compilatore `gcc`.

3 Come Preparare il Codice per GDB (Il flag -g)

Per fare in modo che `gdb` possa associare le righe di codice C ai comandi in linguaggio macchina, devi istruire il compilatore `gcc` ad aggiungere le informazioni di debugging (i simboli) all'eseguibile.

Questo si fa aggiungendo il flag `-g` in fase di compilazione:

```
gcc -g -Wall -Wextra my_program.c -o exe_program
```

Se dimentichi il `-g`, `gdb` riuscirà comunque a eseguire il programma, ma vedrà solo gli indirizzi di memoria e i nomi delle funzioni, senza poter mostrare il codice sorgente riga per riga.

4 Come Avviare GDB

Per lanciare il debugger sull'eseguibile appena compilato, digita nel terminale:

```
gdb ./exe_program
```

Ti comparirà il prompt di GDB (`(gdb)`), pronto a ricevere i tuoi comandi.

5 I Comandi Fondamentali di GDB (La tua cassetta degli attrezzi)

Una volta dentro GDB, puoi usare i seguenti comandi (spesso basta la lettera iniziale come abbreviazione):

A) Esecuzione e Controllo

- `run` (o `r`): Avvia l'esecuzione del programma dall'inizio. Se il programma non contiene errori o fault particolari, arriverà alla fine.
- `break <nome_funzione>` o `break <numero_riga>` (o `b`): Imposta un punto di interruzione (*breakpoint*). Il programma si fermerà prima di eseguire quella riga.
  ATTENZIONE: Non tutte le funzioni possono essere viste da GDB; quelle scritte da te sono pienamente visibili e debuggabili.
  Esempio: `break main` oppure `break my_program.c:45`
- `continue` (o `c`): Riprende l'esecuzione del programma fino al breakpoint successivo o alla fine del programma.
- `quit` (o `q`): Esce da GDB.

B) Esecuzione Passo-Passo (Quando il programma è fermo)

- `next` (o `n`): Esegue la riga corrente e passa alla successiva. Se la riga contiene una chiamata a funzione, la esegue tutta in un colpo solo senza entrare dentro.
- `step` (o `s`): Simile a `next`, ma entra dentro la funzione chiamata alla riga corrente, permettendoti di debuggarla riga per riga.
- `finish`: Termina l'esecuzione della funzione corrente e torna al chiamante, mostrando il valore di ritorno.

C) Ispezione delle Variabili e della Memoria

- `print <variabile>` (o `p`): Stampa il valore attuale di una variabile.
  ATTENZIONE: Se tentate di stampare una variabile fuori scope (non visibile in quel blocco di codice), GDB non riuscirà a vederla.
  Esempio: `print current->data` o `print *mylist`
- `display <variabile>`: Mantiene "fissa" la visualizzazione di una variabile. Ogni volta che il programma si ferma, GDB ti mostrerà automaticamente il suo valore aggiornato.
- `backtrace` (o `bt`): Fondamentale in caso di crash! Ti mostra la pila delle chiamate (*call stack*), ovvero la catena di funzioni che ha portato il programma esattamente nel punto in cui si trova o è crollato.